Compte rendu de réunion n°3 : TER BYZFL

Industrialisation des expériences, génération des courbes et comparaison des agrégateurs

ABBES Samy & AREZKI Célia

25 Mai 2026

1 Contexte général de la semaine

Cette semaine a été consacrée à la mise en place d'une chaîne complète d'expériences reproductibles autour de l'apprentissage fédéré robuste avec *ByzFL*. Après les premiers essais sur MNIST et la clarification théorique des attaques byzantines, l'objectif a été de passer d'exécutions ponctuelles à un pipeline structuré permettant : (i) de définir les scénarios dans des fichiers JSON, (ii) de lancer automatiquement plusieurs expériences, (iii) de sauvegarder les résultats sous forme de fichiers `.csv`, et (iv) de produire des courbes comparatives lisibles pour identifier l'agrégateur le plus robuste.

2 État de l'environnement de travail

2.1 Version Python et bibliothèques

L'environnement d'exécution principal repose sur un *virtual environment* local du projet. Les versions actuellement utilisées sont les suivantes :

- **Python :** 3.13.2
- **PyTorch :** 2.12.0+cpu
- **Torchvision :** 0.27.0+cpu
- **ByzFL :** 0.0.11
- **Pandas :** 3.0.3
- **Matplotlib :** 3.10.9
- **Numpy :** 2.4.6

L'exécution se fait actuellement en mode **CPU uniquement**. La commande de vérification `torch.cuda.is_available()` renvoie `False`. Cela signifie que le projet est pleinement fonctionnel sans GPU, mais que les longues expériences demeurent plus coûteuses en temps de calcul, en particulier lorsque l'on évalue le modèle à chaque round.

2.2 Jeux de données

Deux jeux de données ont été conservés pour les benchmarks :

- **MNIST** : chiffres manuscrits en niveaux de gris, utilisé pour les expériences IID et la majorité des comparaisons d'agrégateurs.
- **FashionMNIST** : vêtements et objets du quotidien en niveaux de gris, utilisé pour enrichir les scénarios non-IID.

Les données sont stockées localement dans le dépôt sous le dossier `data/`. Les scripts ont été configurés pour résoudre ce chemin automatiquement afin d'éviter les dépendances à des emplacements externes.

3 Travail réalisé

3.1 Refactorisation en moteur d'expériences JSON

Le travail principal de la semaine a consisté à restructurer le projet autour d'un moteur d'exécution générique. Au lieu de maintenir plusieurs scripts indépendants pour chaque expérience, nous avons adopté une approche pilotée par configuration :

- `code/experiment_engine.py` : contient la logique de chargement des données, de construction des clients et du serveur, d'application des attaques byzantines, d'exécution des rounds et de sauvegarde des métriques.
- `code/run_experiments.py` : lanceur principal en ligne de commande, capable de lister les scénarios, d'en exécuter un seul ou de lancer toute la batterie d'expériences.
- `code/experiment_configs/common.json` : paramètres globaux partagés par toutes les expériences.
- `code/experiment_configs/scenario_index.json` : index des scénarios disponibles.
- `code/experiment_configs/scenarios/*.json` : définition détaillée de chaque scénario.

Cette architecture permet de modifier un scénario sans toucher au moteur général, ce qui améliore la lisibilité, la reproductibilité et la comparabilité des tests.

3.2 Définition des scénarios expérimentaux

Nous avons structuré les expériences autour de deux types de situations.

1. Expérience de référence sans attaquant Le scénario de base est `iid_clean_average` :

- dataset : MNIST
- distribution : IID
- nombre de clients honnêtes : 4
- nombre de clients byzantins : 0
- agrégateur : Average
- attaque : none

Ce scénario sert de référence pour mesurer le comportement normal du système lorsque tous les clients sont honnêtes.

2. Expériences avec un même attaquant et plusieurs agrégateurs Pour comparer la robustesse des méthodes d'agrégation, nous avons ajouté une famille de scénarios avec le même attaquant `SignFlipping` et un seul client byzantin, afin d'obtenir la configuration demandée : **3 clients honnêtes + 1 client byzantin**. Les scénarios concernés sont :

- `signflipping_Average`
- `signflipping_Median`
- `signflipping_TrMean`
- `signflipping_Krum`

Chaque scénario conserve la même attaque et le même nombre de clients, puis ne change que l'agrégateur. Cette stratégie permet une comparaison propre et directe entre les méthodes.

3.3 Paramètres communs et hyperparamètres

Les paramètres généraux utilisés dans les expériences sont les suivants :

— **seed :** 42
— **device :** cpu
— **model_name :** `cnn_mnist`
— **optimizer_name :** SGD
— **loss_name :** CrossEntropyLoss
— **rounds :** 1000 par défaut
— **batch_size :** 64
— **eval_interval :** 1
— **server_learning_rate :** 0.1
— **client_learning_rate :** 0.01
— **server_weight_decay :** 0.0001
— **client_weight_decay :** 0.0
— **client_momentum :** 0.1

Le choix d'un `eval_interval = 1` permet d'obtenir une courbe d'accuracy à chaque round, ce qui rend les courbes plus détaillées pour l'analyse fine. En revanche, ce choix alourdit le coût d'exécution, puisque le modèle est évalué très fréquemment.

3.4 Agrégateurs et attaques étudiés

Les méthodes d'agrégation étudiées sont :

— **Average** : moyenne simple des gradients.
— **Median** : médiane coordonnée, plus robuste face aux valeurs extrêmes.
— **TrMean** : moyenne tronquée, qui retire les valeurs extrêmes avant la moyenne.
— **Krum** : agrégateur géométrique qui sélectionne le gradient le plus cohérent avec ses voisins.

Les attaques byzantines déjà intégrées dans la plateforme sont :

— **none** : aucun attaquant, cas de référence.
— **SignFlipping** : inversion du signe des gradients.
— **InnerProductManipulation** : attaque paramétrée par `tau`.
— **ALittleIsEnough** : attaque subtile basée sur `tau`.
— **Gaussian** : ajout d'un bruit aléatoire gaussien.

4 Résultats obtenus

4.1 Exécution des expériences

Nous avons d'abord exécuté une série courte de validation sur 20 rounds pour vérifier que la nouvelle architecture fonctionnait correctement. Le lancement s'est fait sur les scénarios suivants :

— `iid_clean_average`
— `signflipping_Average`
— `signflipping_Median`
— `signflipping_TrMean`
— `signflipping_Krum`

Les résultats ont été sauvegardés automatiquement dans `code/experiment_outputs/combined_results.csv`. Ce fichier contient, pour chaque round et chaque scénario, le nom du scénario, le dataset, la distribution, l'agrégateur, l'attaque, le nombre de clients honnêtes, le nombre de clients byzantins, le round, l'accuracy et l'erreur.

4.2 Courbes individuelles par expérience

Chaque scénario a produit sa propre courbe d'accuracy enregistrée dans `code/experiment_outputs/plots`. Ces figures permettent d'observer l'évolution de l'apprentissage round par round pour un scénario précis.

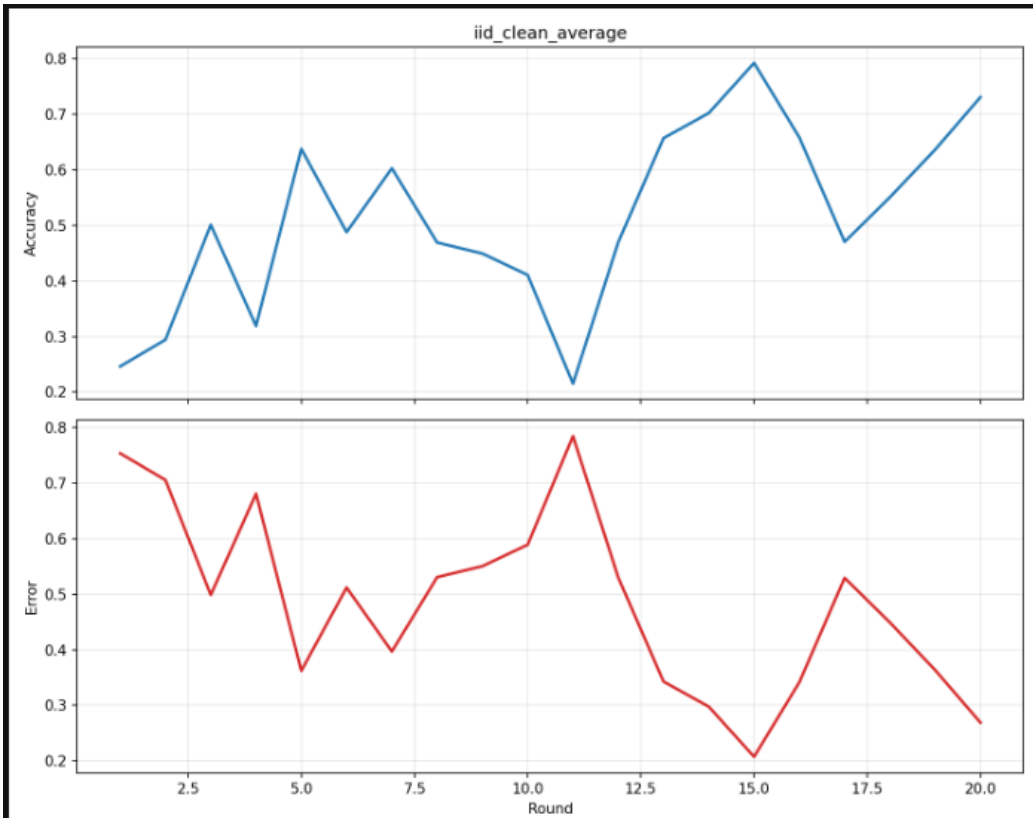


FIGURE 1 – Évolution de l'accuracy pour l'expérience de référence sans attaquant : 4 clients honnêtes, agrégateur Average, dataset MNIST IID.

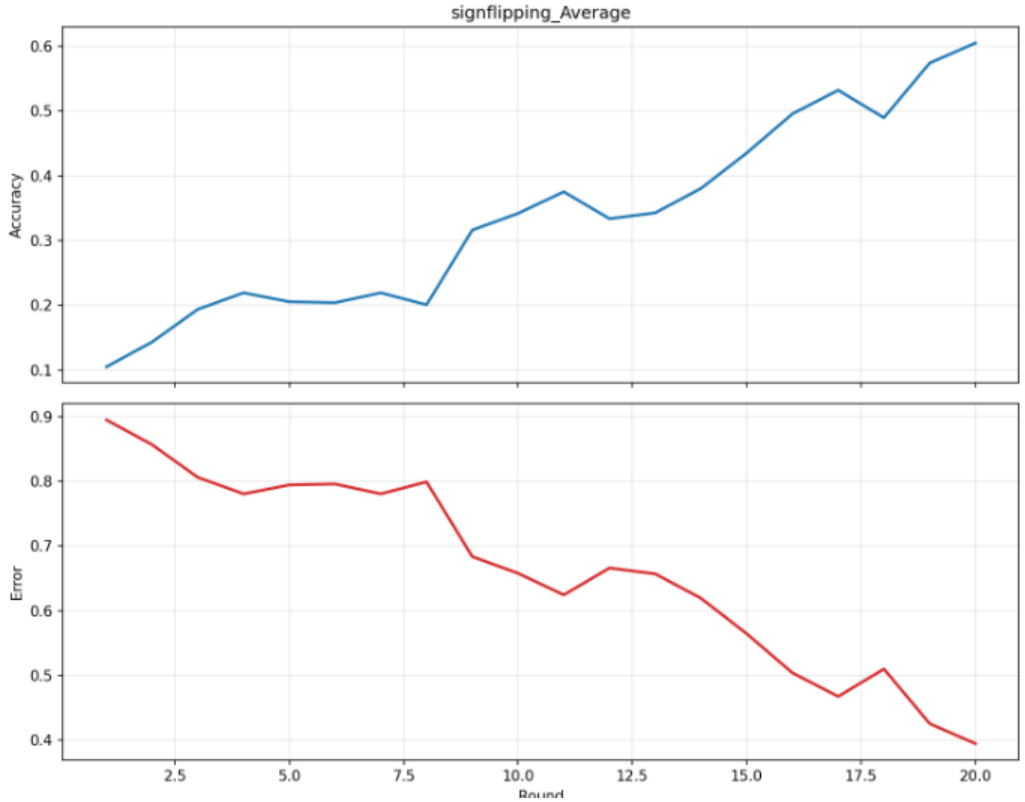


FIGURE 2 – Évolution de l'accuracy avec l'attaque SignFlipping et l'agrégateur Average.

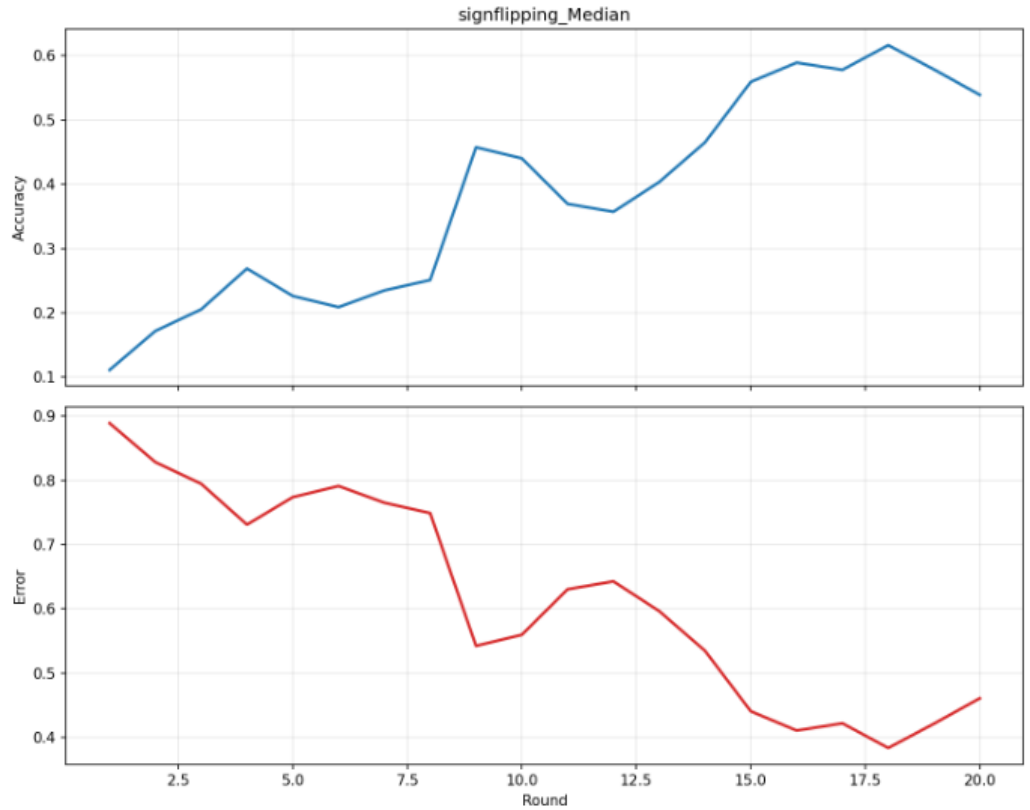


FIGURE 3 – Évolution de l'accuracy avec l'attaque SignFlipping et l'agrégateur Median.

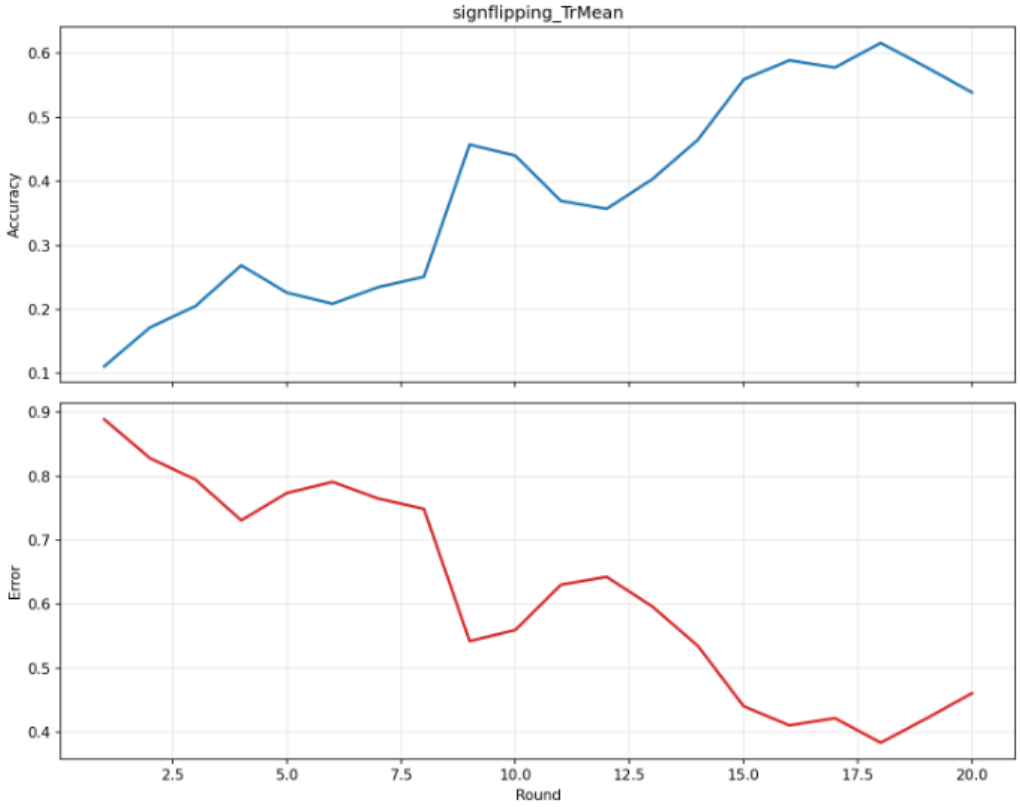


FIGURE 4 – Évolution de l'accuracy avec l'attaque SignFlipping et l'agrégateur TrMean.

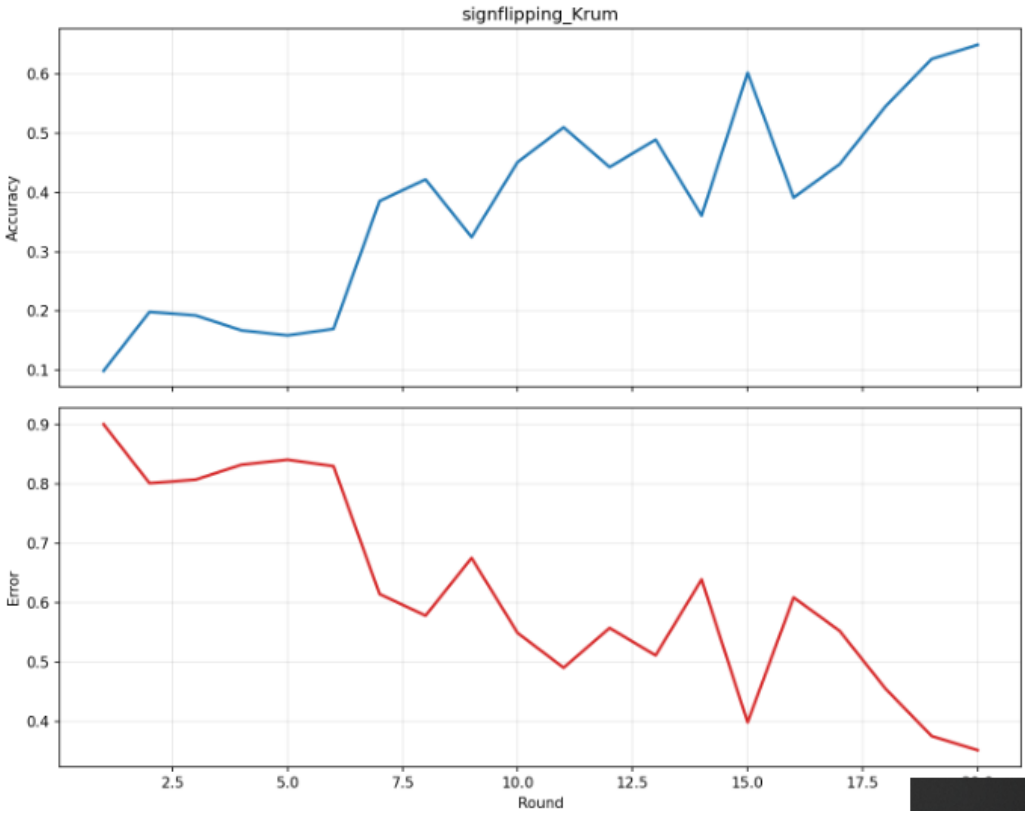


FIGURE 5 – Évolution de l'accuracy avec l'attaque SignFlipping et l'agrégateur Krum.

4.3 Figure globale de comparaison

Pour obtenir une comparaison directe entre toutes les expériences, nous avons créé un second script de visualisation : `code/compare_experiments.py`. Il lit le CSV des résultats et superpose toutes les courbes sur une seule figure. Cette vue d'ensemble permet de voir immédiatement quelle expérience obtient la meilleure accuracy finale.

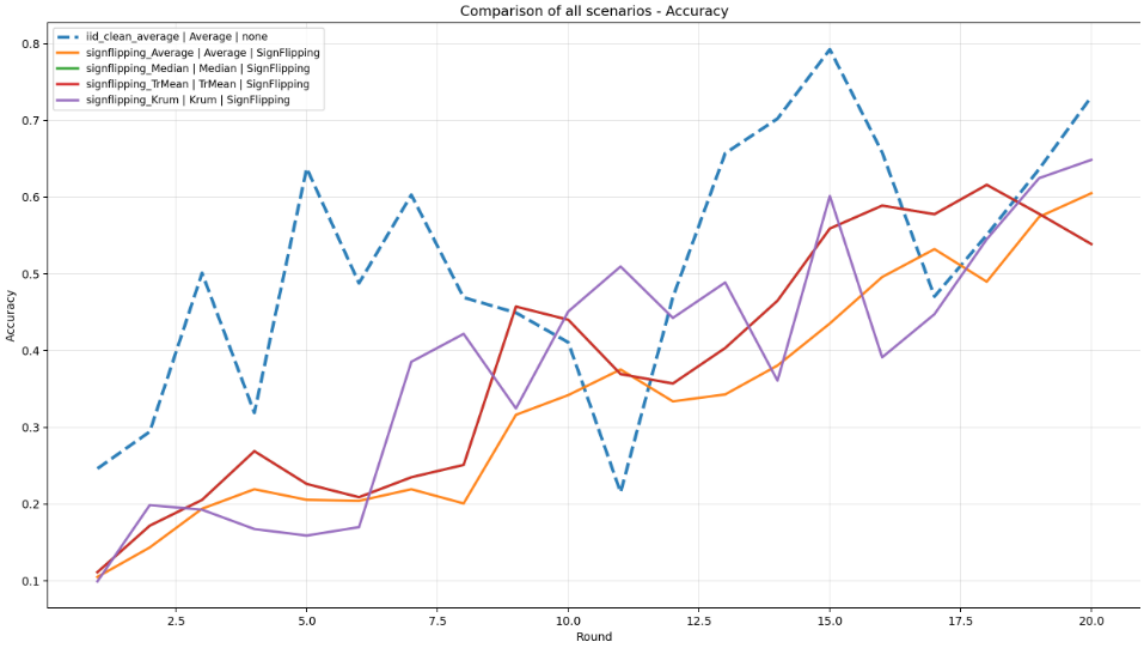


FIGURE 6 – Comparaison globale de toutes les courbes d'accuracy. La baseline saine est comparée aux scénarios attaqués avec SignFlipping et différents agrégateurs.

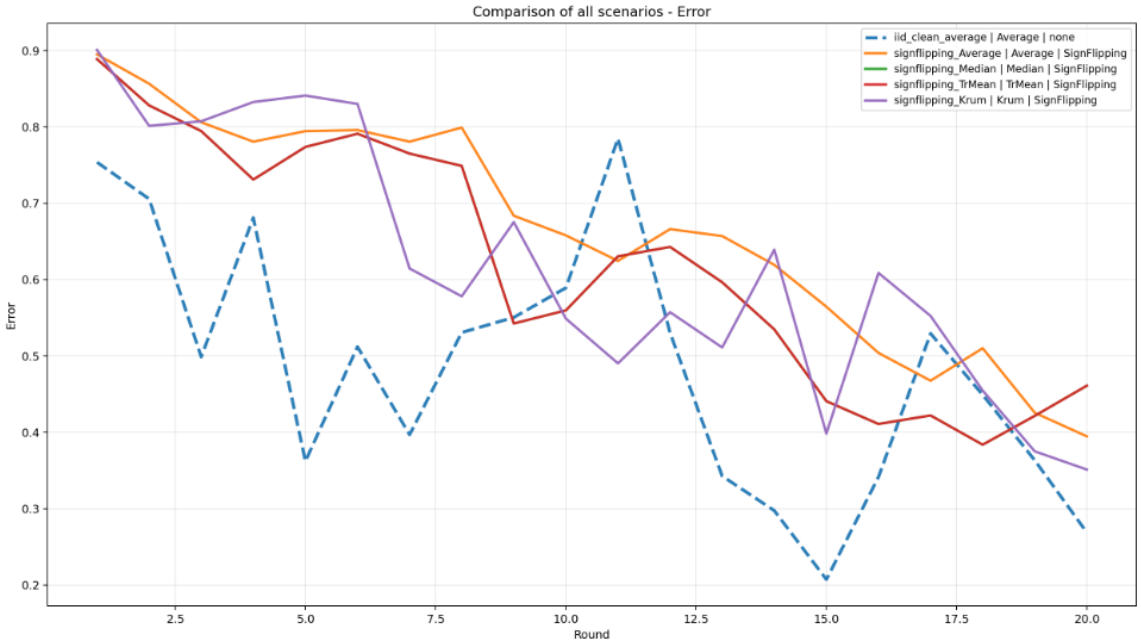


FIGURE 7 – Comparaison globale de toutes les courbes d'erreur. Cette représentation complète la lecture de l'accuracy en montrant l'écart à la performance parfaite.

4.4 Classement observé sur le test court

À partir du fichier `final_round_summary.csv`, nous avons obtenu un classement des scénarios selon leur accuracy finale. Sur le test court effectué cette semaine, l'ordre observé a été :

1. `iid_clean_average`
2. `signflipping_Krum`
3. `signflipping_Average`
4. `signflipping_Median`
5. `signflipping_TrMean`

Ce résultat est cohérent avec l'objectif de robustesse : l'expérience correcte obtient naturellement la meilleure performance, tandis que les agrégateurs robustes comme Krum résistent mieux à l'attaque que la moyenne simple. Les différences entre Median et TrMean montrent que la résistance dépend à la fois du type d'attaque et du mécanisme de filtrage des gradients.

5 Automatisation et reproductibilité

Le pipeline a été conçu pour être relançable sans intervention manuelle lourde. Les commandes suivantes permettent de reproduire les différents niveaux de résultats :

— lister les scénarios : `python code/run_experiments.py -list`

— lancer une expérience précise : `python code/run_experiments.py -scenario iid_clean_average`

— lancer un lot d'expériences : `python code/run_experiments.py -scenario iid_clean_average signflipping_Average ...`

— comparer les résultats globaux : `python code/compare_experiments.py`

Les résultats sont sauvegardés dans un dossier dédié, ce qui permet de conserver les CSV et les figures pour les insérer ensuite dans le rapport final ou dans le mémoire de TER.

6 Difficultés rencontrées et solutions

Plusieurs difficultés ont été rencontrées durant la mise en place :

— **Temps de calcul élevé sur CPU :** les longues expériences sont coûteuses, ce qui a conduit à mettre en place des tests courts pour valider l'architecture avant les exécutions longues.

— **Organisation initiale du code :** la présence de nombreux scripts séparés rendait le projet difficile à maintenir. La migration vers un moteur JSON a résolu ce problème.

— **Comparaison des résultats :** les figures individuelles n'étaient pas suffisantes pour comparer directement les agrégateurs, d'où la création d'un script de comparaison global.

7 Conclusion et prochaines étapes

Cette semaine a permis de franchir une étape importante : le projet n'est plus seulement un ensemble d'essais isolés, mais une véritable plateforme d'expérimentation comparative. Nous avons désormais :

— une base saine de référence sans attaquant ;

— plusieurs scénarios attaqués avec un même adversaire et plusieurs agrégateurs ;

— un format de configuration JSON propre et reproductible ;

— des courbes individuelles et une figure globale de comparaison ;

— un classement objectif des agrégateurs selon l'accuracy finale.

La prochaine étape sera de lancer les expériences longues en 1000 rounds sur les scénarios les plus pertinents, puis d'analyser plus finement la stabilité des courbes et la résistance réelle de chaque agrégateur face aux attaques byzantines.